Ordering Guarantees

keeping events in the order they happened

Watch the 4-min video

In one line: brokers usually order messages only within a partition or queue, so put events that must stay in order under the same key, keep the producer idempotent, preserve per-key order inside consumers, and make handlers tolerant with version numbers when order can still break.

1 The ghost account

A customer creates an account, updates their email, then deletes the account. Your consumer processes the delete first, then the update, and recreates a ghost record. Order matters, and messaging systems only promise it under specific conditions.

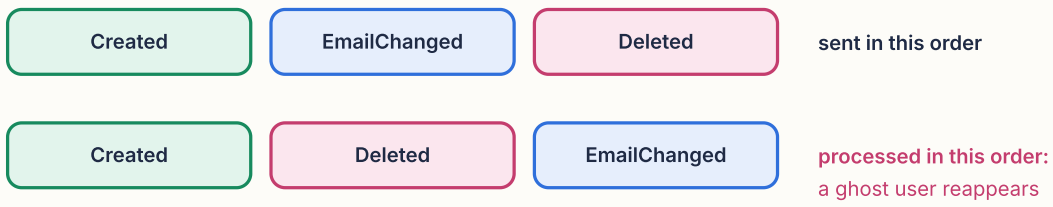


Figure 1. The same three events, processed in the wrong order.

2 What brokers promise

Kafka, Kinesis, Pulsar: order within one partition (shard). **SQS standard:** best effort, no guarantee; **SQS FIFO:** order per message group ID. **RabbitMQ:** order per queue for a single consumer, broken by competing consumers and requeues. **Azure Service Bus:** order per session. There is almost never a global order, and you rarely need one: you need **per-entity** order (per user, per order, per account).

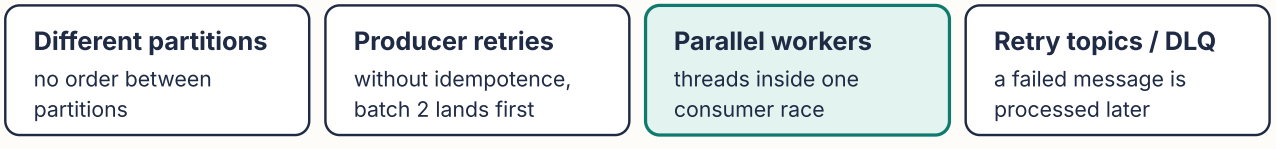


Figure 2. Four ways order gets lost, even on a broker that orders partitions.

3 Getting per-key order end to end

- **Producer:** use the entity ID as the key, so its events share a partition. Enable idempotence so retries can't reorder batches (Kafka keeps order with up to 5 in-flight requests when idempotent).
- **Broker:** don't change the partition count of a keyed topic casually (lesson 26).
- **Consumer:** if you process in parallel, parallelise **by key**, never across a key's events. Libraries like Confluent's parallel consumer do this.
- **Retries:** a message moved to a retry topic falls out of order; for strict cases, block that key until it succeeds.

WORKED EXAMPLE: USER EVENTS

All events for a user are published to the `user-events` topic, keyed by user ID, with an idempotent producer. Every event for user 42 goes to partition 7, in order. The consumer runs 16 worker threads but routes each record to a worker by `hash(user_id)`, so user 42's events are handled by one worker, one at a time, while other users run in parallel. Each event carries a per-user `version`; the handler ignores any event older than the version already stored, as a final safety net.

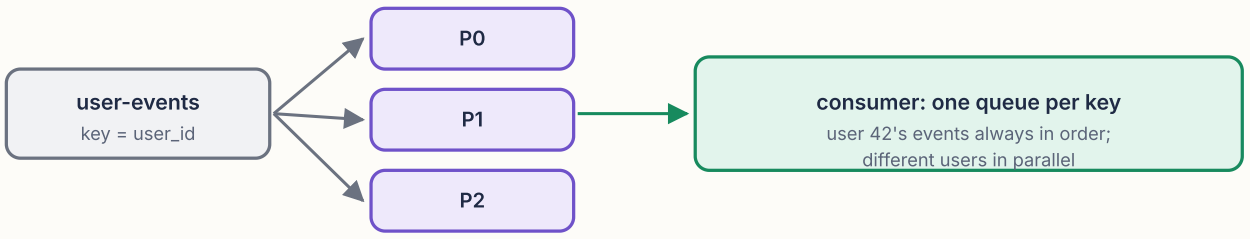


Figure 3. Key by entity, and keep per-key order inside the consumer too.

4 Designing for disorder

When order can't be guaranteed, make handlers tolerant: store a **version** or sequence per entity and drop stale events; use **tombstones** so a late update can't resurrect a deleted record; or send **full state** rather than deltas, so the latest version simply wins.

COMMON MISTAKES

- No key, or a random key, for events that must stay ordered.
- A thread pool consuming one partition without per-key routing.
- One partition for everything 'to be safe'.
- Assuming SQS standard or RabbitMQ with many consumers keeps order.

5 Key terms

Per-partition order: order guaranteed only within a partition

Message group / session: ordering unit in SQS FIFO / Service Bus

Entity version: sequence number to detect stale events

Partition key: decides which partition a record goes to

Head-of-line blocking: one stuck message delays those behind it

6 Interview questions

Q1. Does Kafka guarantee ordering? BASIC

Only within a partition. Records with the same key go to the same partition, so per-key order holds if producers and consumers preserve it.

Q2. How do you keep a user's events in order? BASIC

Key them by user ID, use an idempotent producer, and process one user's events sequentially in the consumer.

Q3. You added a thread pool to a consumer and now see out-of-order updates. Fix it. SCENARIO

Route records to threads by key so each key is processed by one thread in order, and commit offsets only up to the lowest completed record.

Q4. Ordering can't be guaranteed across two topics. How do you stay correct? SCENARIO
Version every entity, ignore events with a lower version than stored, use tombstones for deletes, or merge into one topic keyed by entity.

Q5. Can producer retries reorder messages in Kafka? TRICKY
Without idempotence, with several in-flight requests, a failed batch retried after a later batch succeeded lands out of order. Idempotence fixes this.

Q6. Why is global ordering usually a bad requirement? TRICKY
It forces one partition and one consumer, so throughput is capped at one machine; per-entity ordering almost always suffices.

Next: SD-I-008 · Delivery Semantics backendroadmap.com